attributes.

Before and After Callbacks

As their names suggest, before and after callbacks are invoked before or after an action. Rails maintains two chains of callbacks for each controller. When a controller is about to run an action, it executes all the callbacks on the before chain. It executes the action before running the callbacks on the after chain.

Callbacks can be passive, monitoring activity performed by a controller. They can also take a more active part in request handling. If a before action callback returns `false`, then processing of the callback chain terminates and the action is not run. A callback may also render output or redirect requests, in which case the original action never gets invoked.

We saw an example of using callbacks for authorization in the administration part of our store example in *Iteration I3: Limiting Access*. We defined an authorization method that redirected to a login screen if the current session didn't have a logged-in user. We then made this method a before action callback for all the actions in the administration controller.

Callback declarations also accept blocks and the names of classes. If a block is specified, it'll be called with the current controller as a parameter. If a class is given, its `filter` class method will be called with the controller as a parameter.

By default, callbacks apply to all actions in a controller (and any subclasses of that controller). You can modify this with the `:only` option, which takes one or more actions on which the callback is invoked, and the `:except` option, which lists actions to be excluded from callback.

The `before_action` and `after_action` declarations append to the controller's chain of callbacks. Use the variants `prepend_before_action` and `prepend_after_action` to put callbacks at the front of the chain.

After callbacks can be used to modify the outbound response, changing the headers and content if required. Some applications use this technique to perform